

Relatório Técnico: Comparação de Desempenho entre Árvore Binária de Busca e AVL

Rogers Rocha Carvalho

April 20, 2026

Abstract

Este relatório baseia-se na execução do primeiro projeto da disciplina de Estrutura de Dados II, focando nos pontos cruciais para o entendimento do problema e de sua resolução. O desenvolvimento foi realizado em linguagem C, utilizando estruturas hierárquicas como árvores binárias de busca e árvores AVL, que garantem o equilíbrio automático da estrutura. Por meio de testes com diversos fluxos de informações, incluindo casos com entradas já ordenadas, validou-se as funções de inserção e busca, comprovando que ambas são ideais para sistemas que buscam eficiência máxima no controle de dados.

1 Introdução

Este relatório técnico apresenta uma análise comparativa do desempenho entre Árvore Binária de Busca (ABB) e Árvore AVL, pois, com o aumento da quantidade de dados, surgiram estruturas como listas encadeadas e árvores para deixar esses processos mais rápidos. Mesmo assim, quando o volume de dados é muito grande, nem todas essas estruturas conseguem oferecer o melhor desempenho nas buscas. Essas estruturas de dados são muito usadas para organizar e lidar com informações de forma eficiente. Elas foram implementadas em C e testadas principalmente nas operações de inserção e busca.

A finalidade central deste trabalho consistiu em examinar o efeito do balanceamento automático da Árvore AVL em relação ao comportamento convencional da ABB, sobretudo em cenários com elevados volumes de dados e entradas fortemente ordenadas. Nessas circunstâncias, a ABB tende a se degradar, adotando um comportamento linear e prejudicando sua performance. Para a análise, foram desenvolvidas rotinas específicas para as operações fundamentais de ambas as estruturas, assegurando a precisão dos experimentos de desempenho.

O trabalho descrito neste relatório emprega essas estruturas com o objetivo de viabilizar o armazenamento e a manipulação de um clube do livro, associando árvores a assinantes, livros e assinaturas, lista dinâmica à forma de assinatura, e lista estática aos gêneros dos livros. Desenvolveu-se duas versões do sistema, cada uma utilizando um tipo distinto de árvore, aplicando operações essenciais dessas estruturas: inserção, busca, remoção e exibição. Além disso, foram realizados testes comparativos de tempo em dois cenários, inserção e busca, para cada uma das árvores binárias implementadas, com a finalidade de obter resultados que evidenciem as diferenças entre elas.

2 Seções Específicas

2.1 Árvore Binária de Busca

As (ABB) funcionam como árvores comuns, mas com uma regra de ouro que agiliza a procura: qualquer nó à esquerda deve ser menor que o pai, enquanto o da direita deve ser maior. Essa estratégia permite descartar metade das opções a cada passo, já que só precisa seguir por um dos caminhos. Assim, encontra-se o que deseja, ou descobre que não existe, fazendo muito menos comparações do que em uma lista simples.

2.2 Árvore AVL

Por outro lado, a Árvore AVL funciona com regras adicionais, as quais envolvem a altura e o balanceamento. A altura mede o caminho até o nó mais distante, começando do zero nas folhas e subindo. Já o fator de balanceamento é a diferença de altura entre os lados esquerdo e direito, considerando -1 para nós vazios. Funciona como um alerta: se o valor chegar a 2, a árvore pesou para a esquerda; se cair para -2, o peso está na direita, sinalizando que é hora de reajustar a estrutura.

2.3 Structs

2.3.1 Struct Data

- Dia, mes e ano.

2.3.2 Struct Assinante

- Cpf;
- Nome;
- Endereco;
- Data;
- Filho à esquerda;
- Filho à direita.

2.3.3 Struct Livro

- Isbn;
- Titulo;
- Autor;
- Editora;
- Edicao;
- Ano_publica;
- Tipo_encadern;
- Valor_mensal;
- Valor_anual;
- Próxima forma de assinatura.

2.3.4 Struct Genero

- Codigo;
- Nome;
- Colecao_livros.

2.3.5 Struct forma ass

- `Codigo;`
- `Qtd_livros_mensais;`
- `Qtd_generos_mensais;`
- `Generos_escolhidos;`
- `Tipo_encarden;`
- `Valor_mensal;`
- `Valor_manual;`
- Próxima forma de assinatura.

2.3.6 Struct Assinatura

- `Cpfassinante;`
- `Codigo_forma;`
- `Dataassinatura;`
- `Datavencimento;`
- `Valor;`
- `Filho à esquerda;`
- `Filho à direita.`

2.4 Funções

2.4.1 Funções de Alocação

- Nas árvores: A função `alocar` recebe os atributos dos seus respectivos tipos - Assinante, Livro ou Assinatura -, cria o nó, aloca a memória necessária, inicializa os ponteiros e, caso seja uma árvore AVL, a altura, retornando o nó logo em seguida para dentro da raiz onde ele foi chamado.
- Na lista estática: A função `criar_genero` inicializa uma estrutura do tipo `Genero`, atribuindo o código do livro e copiando o nome do gênero para o campo correspondente. Além disso, define a coleção de livros como vazia (NULL), indicando que o gênero ainda não possui livros associados. Não realiza alocação dinâmica, apenas retorna a struct já preenchida.
- Na lista dinâmica: A função `alocar_formaassinatura` realiza a alocação dinâmica de memória para um nó do tipo `forma ass`. Após a alocação, inicializa todos os campos com valores padrão (zeros para atributos numéricos e NULL para ponteiros), garantindo um estado inicial consistente. Em caso de falha na alocação, exibe uma mensagem de erro. O ponteiro para o próximo elemento da lista também é inicializado como NULL, indicando que o nó ainda não está encadeado. Retorna o ponteiro para a estrutura alocada.

2.4.2 Funções de Cadastro

- Nas árvores: A função `cadastrearassinante` e `cadastrearlivro` recebem a árvore por referência, coleta os seus respectivos dados, aloca um novo nó e preenche seus campos internamente. Só efetiva o cadastro se todas as etapas forem bem-sucedidas. No caso da função `cadastrearassinatura`, também valida a existência do assinante e da forma de assinatura escolhida. Não retornam valor.

- Na lista dinâmica: A função `cad_forma_assinatura` recebe a lista por parâmetro, interage com o usuário para a coleta de dados e realiza a alocação dinâmica do nó e de seu vetor interno de gêneros. O preenchimento dos campos ocorre internamente, incluindo o cálculo automático do valor anual. O cadastro só é efetivado e inserido na lista principal se todas as etapas de alocação e leitura forem bem-sucedidas; caso contrário, a memória é liberada para evitar vazamentos. Retorna a lista atualizada com o novo nó inserido.

2.4.3 Funções de Inserção

- Nas árvores: A função de inserir recebe a raiz da sua respectiva árvore por referência e o novo nó já alocado por parâmetro. A inserção ocorre de forma recursiva, comparando o CPF (isbn no livro) do novo nó com os nós existentes para determinar sua posição à esquerda ou à direita, mantendo a propriedade da árvore binária de busca e da AVL. O cadastro só é efetivado se o CPF (isbn no livro) for exclusivo; caso seja detectada uma duplicidade, o nó é liberado da memória e a operação falha. Retorna um valor inteiro indicando o sucesso ou a falha da inserção. Na AVL, chama-se a função `balancear_assinante`, `balancear_ass` ou `balancear_liv`.
- Na lista estática: A função `inserir_genero` recebe o vetor de estruturas, o contador de elementos por referência e o novo dado já preenchido por parâmetro. A inserção ocorre de forma sequencial, copiando a struct para a primeira posição disponível no índice indicado pelo contador. O cadastro só é efetivado se a quantidade atual for inferior ao limite máximo permitido pelo sistema; caso o limite seja atingido, a operação falha e uma mensagem de erro é exibida. Retorna um valor inteiro indicando o sucesso ou a falha da inserção.
- Na lista dinâmica: A função `inserir_na_lista` recebe o ponteiro para o início da lista e o novo nó já alocado por parâmetro. A inserção é realizada de forma direta no início da estrutura, onde o novo elemento passa a apontar para a lista atual e se torna a nova cabeça da corrente. O cadastro só é efetivado se o novo nó não for nulo, garantindo a integridade da conexão. Retorna o ponteiro atualizado que representa o novo início da lista.

2.4.4 Funções de Busca

- Na árvore: A função `buscar_assinante` recebe como parâmetros de entrada a raiz da árvore e uma string contendo o CPF para a localização. A operação é realizada de forma recursiva, utilizando uma variável local para garantir uma saída única ao final do processamento. A busca ocorre através da comparação do CPF informado com a chave do nó atual: caso sejam idênticos, o endereço é capturado; se o valor buscado for menor, a função explora a subárvore esquerda; caso contrário, segue pela direita. A saída consiste no retorno do ponteiro para o assinante encontrado ou o valor nulo caso o registro não exista na estrutura.
- Na lista dinâmica: A função `buscar_forma` recebe como parâmetros de entrada o ponteiro para o início da lista e o código de identificação desejado através de passagem por valor. A localização do registro ocorre de forma iterativa, utilizando uma variável local para consolidar uma saída única. A busca percorre a estrutura sequencialmente, comparando o código informado com cada nó até encontrar a correspondência ou atingir o fim da lista. A saída retorna o endereço da forma de assinatura localizada ou nulo em caso de ausência.

2.4.5 Funções de Remoção

- Na árvore: A função `remover` recebe a raiz da sua respectiva estrutura por referência e o CPF do registro a ser excluído, operando de forma recursiva com saída única. A lógica contempla três cenários: a remoção de um nó folha, de um nó com apenas um filho e de um nó com dois filhos, neste último, realiza a substituição pelo maior elemento da subárvore esquerda. A operação é finalizada com a liberação da memória do nó removido e retorna 1 em caso de sucesso ou 0 se o registro não for localizado. Na AVL, chama-se a função `balancear_assinante` para a árvore dos assinantes, ou `balancear_ass` para a árvore das assinaturas.

2.4.6 Função de Balanceamento e Altura

- **Altura:** recalcula a altura de um nó com base na altura de seus filhos.
- **Fator de balanceamento:** calcula o fator de balanceamento como diferença entre alturas do filho esquerdo e direito, e usa o operador ternário para evitar erros de segmentação.
- **rot_Esq / rot_dir:** rodam a árvores para deixá-la balanceada.
- **Balanceamento:** verifica o fator de balanceamento e aplica rotações conforme necessário. Retorna o ponteiro para a nova raiz da subárvore devidamente equilibrada.

2.4.7 Funções de Mostrar

- Mostram os dados referentes a assinaturas cadastradas, assinatura de uma determinada forma, gêneros cadastrados, gêneros assinados, todos os livros de um determinado gênero e datas de vencimento e uma assinatura.
- Exemplos: `mostrar_assinantes`, `mostrar_assinaturas` e `mostrar_generosassinados`.

2.5 Teste

Nos ensaios relacionados à inserção de uma quantidade significativa de elementos em uma árvore de livros, observa-se o comportamento do desempenho da Árvore Binária de Busca diante de diferentes ordenações dos dados de entrada. Para esse experimento, foram inseridos aproximadamente 4.000.000 de registros, considerando três cenários distintos: dados previamente ordenados, distribuídos de forma aleatória e organizados de maneira desordenada. Converteram-se os tempos ao utilizar CLOCKS PER SEC da biblioteca `time.h`.

Para analisar o desempenho das operações de leitura, tanto a árvore binária de busca quanto a árvore AVL foram previamente preenchidas com 80.000 registros de livros pertencentes a um único gênero, sendo então mensurado o tempo necessário para a varredura completa da estrutura, com o objetivo de localizar e contabilizar todos os elementos armazenados. O experimento foi repetido 30 vezes consecutivas, possibilitando o cálculo da média aritmética dos tempos obtidos com o intuito de garantir maior confiabilidade nos resultados. O tempo do teste de busca foi coletado da mesma forma do teste de inserção, só que multiplicou-se seus tempos de busca por 100 para obter milissegundos e ficarem de mais fácil entendimento.

2.5.1 Especificação de Hardware

Atributo	Especificação
Modelo	Samsung Book
Processador	11th Gen Intel(R) Core (TM)
Memória RAM	16 GB DDR4
Armazenamento	256 GB
Sistema Operacional	Windows 11
Versão do GCC	13.4.0

Table 1: Especificações técnicas da máquina e softwares utilizados nos experimentos.

3 Resultados

3.1 Teste de Inserção

Conforme apresentado na Tabela 2, os resultados indicam que os tempos médios de inserção permaneceram bastante próximos entre si. A inserção com dados ordenados apresentou tempo médio de 0,426231 segundos, enquanto a inserção aleatória registrou 0,432246 segundos. Já o cenário com dados desordenados resultou em um tempo médio de 0,438942 segundos. Essa proximidade entre os valores

evidência que, mesmo com um volume elevado de elementos, a estrutura manteve um desempenho relativamente estável, sem variações abruptas entre os diferentes padrões de entrada.

Ordem de Inserção	Tempo médio (s)
Ordenada	0,426231 s
Aleatória	0,432246 s
Desordenada	0,438942 s

Table 2: Tempo médio de inserção na Árvore Binária de Busca

Dessa forma, é possível inferir que, nas condições avaliadas, a ordem dos dados não exerceu influência significativa sobre o custo de inserção, ao menos no contexto experimental adotado. Esse comportamento sugere que a implementação utilizada conseguiu lidar de maneira eficiente com grandes volumes de dados, mantendo consistência no tempo de execução independentemente da organização inicial dos elementos inseridos.

Contudo, na tabela 3, os achados demonstram que os períodos médios de inclusão exibiram oscilações mais acentuadas conforme a configuração dos dados de entrada. A inserção de registros sequenciais obteve uma média de 0,414893 segundos, ao passo que a entrada aleatória computou 0,438087 segundos. Em contrapartida, o panorama com dados inversamente ordenados demandou um tempo médio substancialmente maior, atingindo 0,666450 segundos.

Ordem de Inserção	Tempo médio (s)
Ordenada	0,414893
Aleatória	0,438087
Desordenada	0,666450

Table 3: Tempo médio de inserção na Árvore AVL

Estes dados indicam que, embora o auto-balanceamento da AVL garanta eficiência de $O(\log n)$, o overhead das operações de rotação afeta o tempo de execução de forma mais contundente dependendo da ordem dos registros, sendo o caso desordenado o mais impactante. Esse impacto ocorre porque inserções que provocam maior desbalanceamento exigem um número maior de rotações para restauração da propriedade AVL. Assim, o custo computacional das rotações se torna um fator relevante no tempo total de execução, principalmente em sequências de entrada menos estruturadas.

3.2 Teste de Busca

Conforme apresentado na Tabela 4, os resultados relacionados ao tempo de busca evidenciam um desempenho bastante eficiente em ambas as estruturas analisadas. A árvore binária de busca apresentou tempo médio de aproximadamente 0,2315 milissegundos, enquanto a árvore AVL registrou cerca de 0,2084 milissegundos.

Estruturas de dados avaliadas	Livros buscados	Tempo médio de busca
Árvore binária de busca	80.000	0.002315s
Árvore AVL	80.000	0.002084s

Table 4: Comparação do tempo médio de busca entre a Árvore Binária de Busca e a AVL

Observa-se que os valores são extremamente próximos, indicando que, para o cenário avaliado, ambas as estruturas são capazes de realizar operações de leitura com elevada rapidez. Ainda assim, a árvore AVL demonstrou uma leve vantagem, possivelmente em decorrência de seu balanceamento automático, o que contribui para uma altura mais controlada e, conseqüentemente, para uma travessia mais eficiente. Essa pequena diferença reforça que, embora ambas apresentem desempenho satisfatório em operações de busca, estruturas balanceadas tendem a oferecer maior previsibilidade e consistência no tempo de acesso, especialmente à medida que o volume de dados cresce.

4 Conclusão

Curiosamente, a AVL foi mais veloz que a ABB no cenário ordenado (0,41s vs 0,42s), pois, em uma ABB comum, a inserção de dados já ordenados cria uma árvore totalmente degenerada, sendo similar a uma lista, aumentando a profundidade e o tempo de busca pelo local de inserção. Então manter a árvore AVL balanceada compensou o tempo gasto para localizar a posição de cada novo nó, mesmo com o custo das rotações.

Todavia, ao se olhar para o cenário desordenado, o ponto de discrepância é ainda maior, uma vez que a ABB manteve uma média estável (0,43s), ao passo que a AVL foi 50% (0,66s) mais lenta que a árvore binária de busca. Isso pode ter ocorrido porque, em uma inserção desordenada, a AVL é obrigada a realizar sucessivas rotações simples e duplas para manter-se balanceada. Logo, o esforço computacional para rotacionar os nós superou o benefício de busca que o balanceamento oferece.

Por fim, pode-se concluir que a ABB apresentou tempos muito próximos em todos os cenários ($\approx 0,42s$ a $0,43s$), indicando que, para este volume de dados de 4.000.000 de registros, a estrutura simples da ABB não foi penalizada pela altura da árvore. Já a AVL mostrou-se sensível ao padrão de entrada, ganhando em eficiência de busca posterior, mas o seu custo de construção é nitidamente mais alto quando a sequência de entrada exige reestruturações constantes. Isto é, paga-se mais caro no momento da inserção para garantir que, no futuro, qualquer operação de busca seja extremamente rápida ($O(\log n)$) ao usar-se a árvore AVL.